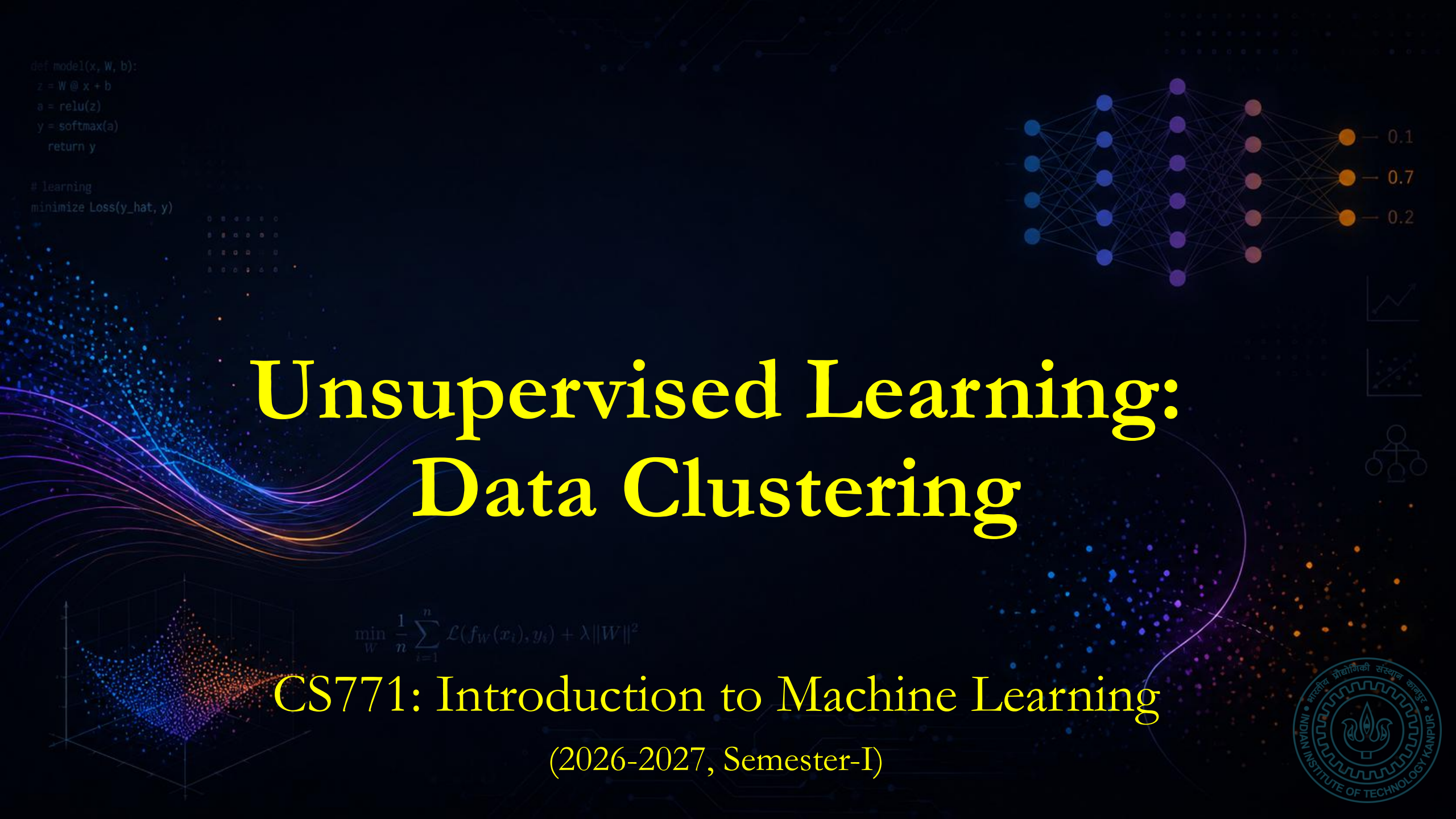


Unsupervised Learning: Data Clustering

CS771: Introduction to Machine Learning

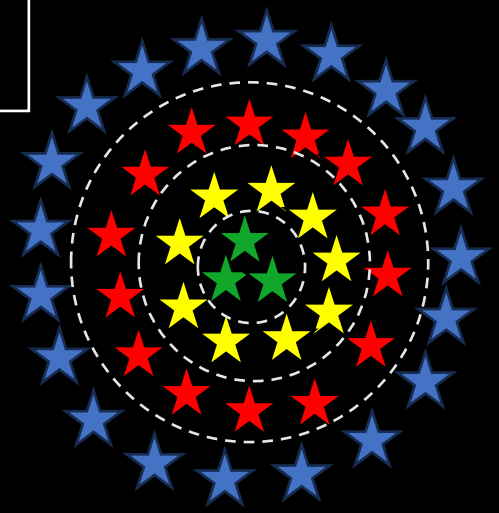
(2026-2027, Semester-I)

Classification without Ground Truth Labels?

Classification partitions inputs but we have some reference for what the groups are

$$\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N}$$

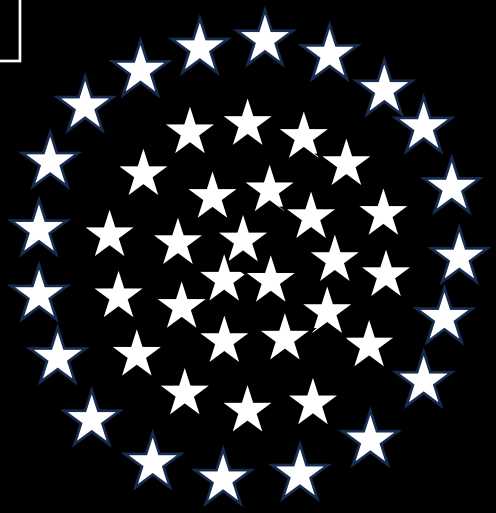
Training labels become our reference



Clustering also partitions inputs but we have no reference for what the groups are

$$\mathcal{D} = \{x_n\}_{n=1}^{N}$$

No training labels, thus no reference!



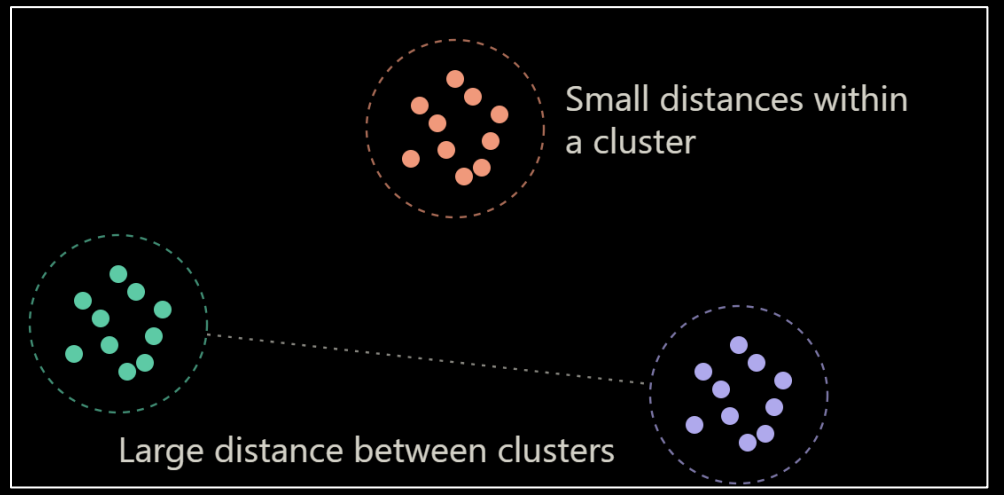
Clustering requires specifying what defines a group/cluster

भारतीय प्रौद्योगिकी संस्थान कानपुर
INDIAN INSTITUTE OF TECHNOLOGY KANPUR

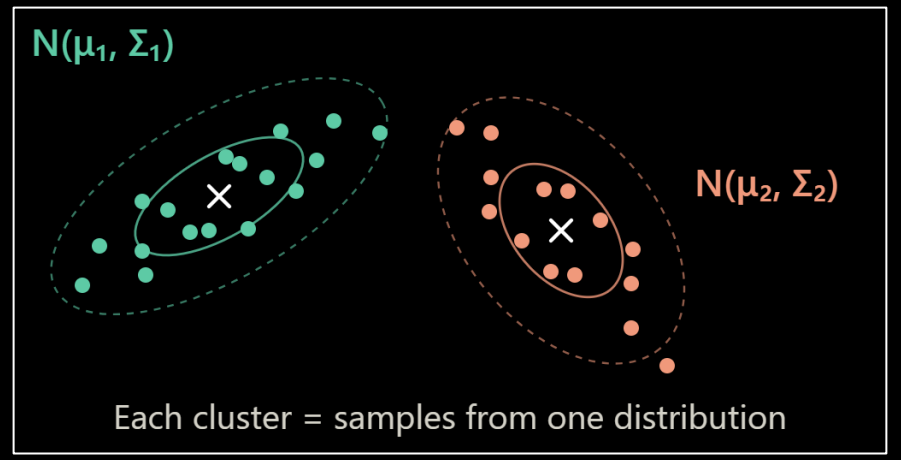
What is a Cluster?

Several notions possible, e.g.,

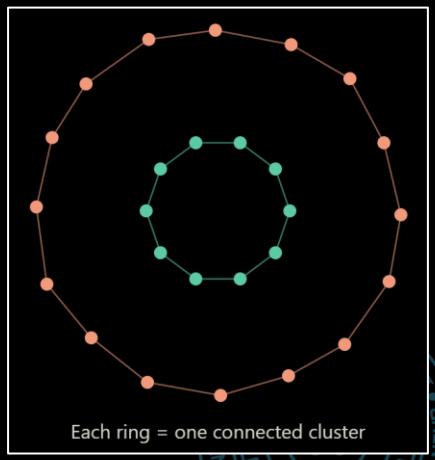
- A set of inputs with high similarity with each other



- A set of inputs generated from the same distribution



- A set of inputs connected with each other



A Simple Algorithm for Clustering

Important: This simple version of K-means clustering has limitations (will see) but many of them are fixable using other representations of the input ($x_n \rightarrow \phi(x_n)$) and/or distances other than Euclidean



K-means clustering. Assumes K mean locations

Pick the K initial means (e.g., randomly or more carefully initialized)

Perform the following 2 steps in an alternating fashion until not converged

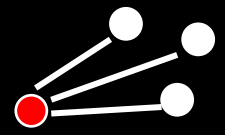
- Assign each input to the current closest mean

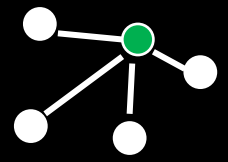
$$z_n = \underset{k}{\operatorname{argmin}} \| x_n - \mu_k \|^2$$

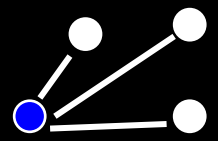
- Recompute (update) the means

$$\mu_c = \frac{1}{N_c} \sum_{n:z_n=c} x_n$$

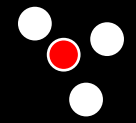
Assigned to current closest mean

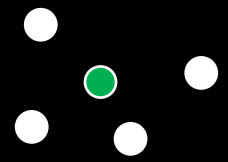






The recomputed means







K-means and Quantization Loss

A way of data compression: N objects approximated by K means

Assigning an input to its closest mean is a form of quantization

$$x_n \approx \mu_{z_n}$$

$z_{nc} = 1$ if x_n is assigned to cluster c, and 0 otherwise

Using a one-hot encoding for $z_n = [z_{n1}, z_{n2}, \ldots, z_{nK}]$ the total quantization loss of all the inputs $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$

Assuming squared error for quantization

$$\ell(\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{Z}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

$N \times D$ $N \times K$ $K \times D$

$$= \|\boldsymbol{X} - \boldsymbol{Z}\boldsymbol{\mu}\|_F^2$$

(K-means As matrix factorization of X)

Exercise

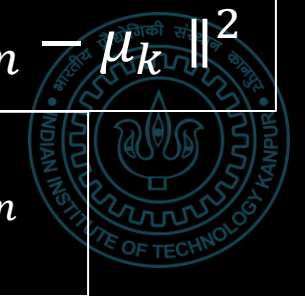
The K-means clustering algo is ALT-OPT for minimizing $\ell(\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{Z})$

Given current $\boldsymbol{\mu}$, solve for $\boldsymbol{Z} = \{\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, z_N\}$

$$z_n = \underset{k}{\operatorname{argmin}} \| x_n - \mu_k \|^2$$

Given current $\boldsymbol{Z}$, solve for $\boldsymbol{\mu} = \{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \mu_K\}$

$$\mu_c = \frac{1}{N_c} \sum_{n:z_n=c} x_n$$



Convergence of K-means algorithm

$$\sum_{n=1}^{N} \|x_n - \mu_{z_n}\|^2$$

The loss function of K-means: $\ell(\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{Z}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \|x_n - \mu_k\|^2$

If using ALT-OPT, when we update Z from $Z^{(t-1)}$ to $Z^{(t)}$

$$\ell\left(\boldsymbol{X}, \boldsymbol{\mu}^{(t-1)}, \boldsymbol{Z}^{(t)}\right) \leq \ell\left(\boldsymbol{X}, \boldsymbol{\mu}^{(t-1)}, \boldsymbol{Z}^{(t-1)}\right)$$

because $Z^{(t)} = \arg\min_{Z} \ell\left(\boldsymbol{X}, \boldsymbol{\mu}^{(t-1)}, \boldsymbol{Z}\right)$

Likewise when we update μ from $\mu^{(t-1)}$ to $\mu^{(t)}$

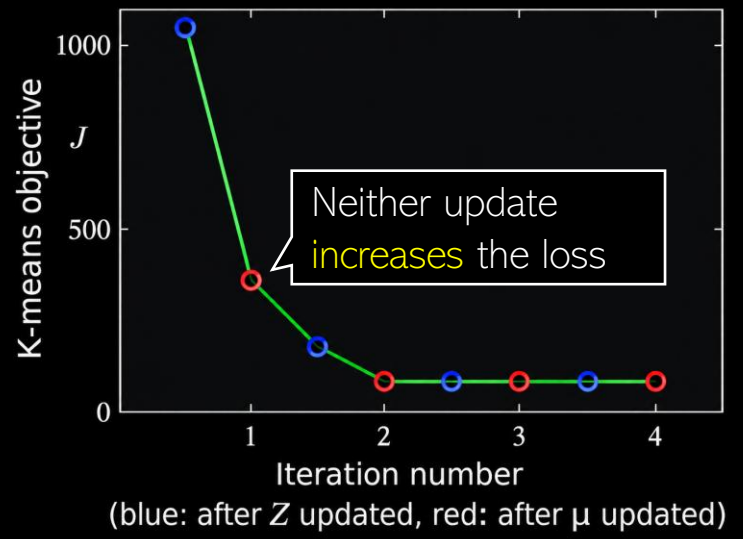
$$\ell\left(\boldsymbol{X}, \boldsymbol{\mu}^{(t)}, \boldsymbol{Z}^{(t)}\right) \leq \ell\left(\boldsymbol{X}, \boldsymbol{\mu}^{(t-1)}, \boldsymbol{Z}^{(t)}\right)$$

because $\boldsymbol{\mu}^{(t)} = \arg\min_{\boldsymbol{\mu}} \ell\left(\boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{Z}^{(t)}\right)$

Choice of initial means may also matter (bad initialization can give poor clusters) a lot for some dataset



Instead of randomly chosen initial mans, it is possible to use smarter initialization. K-means++ is such a method (ensures that initial means are far from each other)



Hard vs Soft Clustering

Standard K-means makes a hard assignment of inputs to clusters

Since z_n is discrete, x_n belongs to a cluster completely or doesn't belong to it at all

$$z_n = \underset{k}{\operatorname{argmin}} \| x_n - \mu_k \|^2$$



When clusters overlap, a soft assignment is preferable

Soft/fuzzy K-means is a popular heuristic

An estimate of probability of x_n belonging to each cluster

Hard K-means: each iteration

$$z_n = \operatorname{argmin}_k \|x_n - \mu_k\|^2 \quad \forall n$$

Only inputs with $z_{nk} = 1$ contribute to μ_k

$$\mu_k = \frac{1}{N_k} \sum_{n:z_n=k} x_n = \frac{\sum_{n=1}^{N} z_{nk} x_n}{\sum_{n=1}^{N} z_{nk}}$$

Converting distance into an estimate of probability

Soft/Fuzzy K-means: each iteration

$$z_{nk} = \frac{\exp(-\|x_n - \mu_k\|^2)}{\sum_{\ell=1}^{K} \exp(-\|x_n - \mu_\ell\|^2)} \quad \forall n, k$$

All N inputs contribute to μ_k since $z_{nk} > 0 \ \forall k$

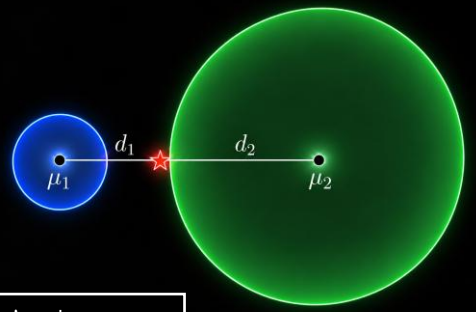
$$\mu_k = \frac{\sum_{n=1}^{N} z_{nk} x_n}{\sum_{n=1}^{N} z_{nk}}$$

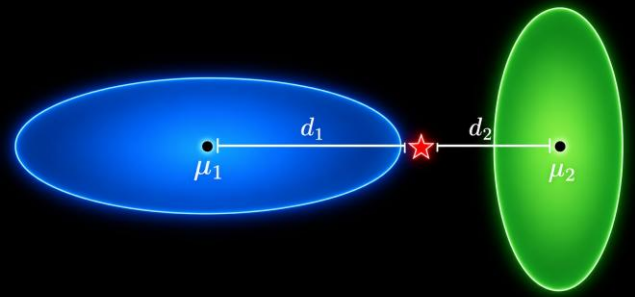
K-means clustering: Cluster Shapes Matter!

Standard K-means if using Euclidean distance typically struggles if the clusters are not "round" and equisized

Recall that LwP also had the same issue

Using a different feature representation or kernels or a non-Euclidean distance can help in such cases

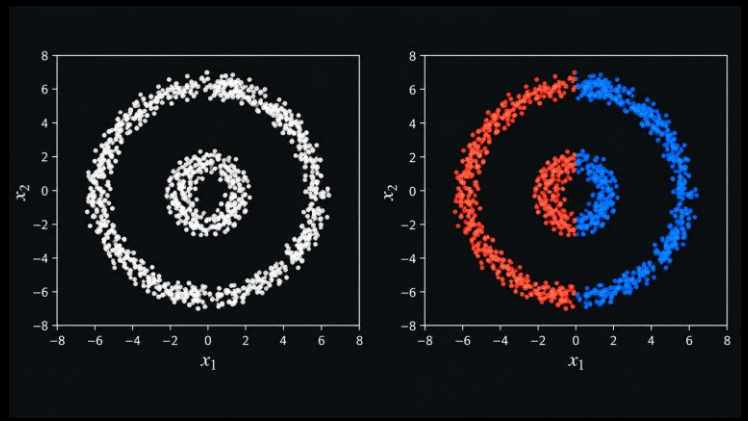


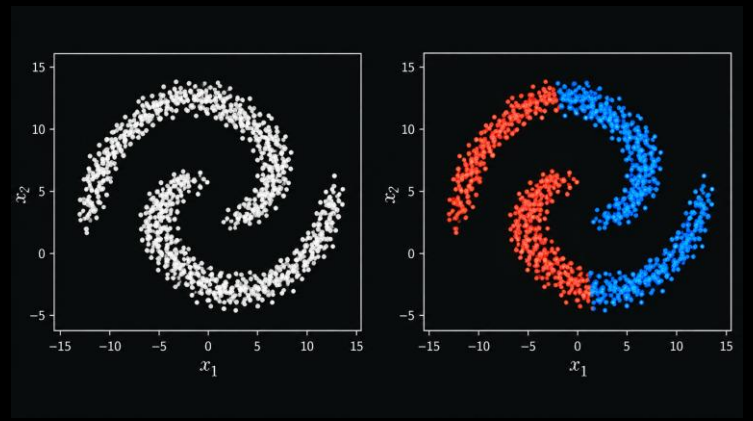


The K-means algorithm isn't the issue; the features/distance used is

Vanilla K-means would fail in all these cases

Kernel K-mean, Deep Embedded Clustering (DEC), etc are such variants that make K-means very powerful





Evaluating Clustering Algorithms

One criterion: Are the clusters compact and well-separated?

Silhouette score is one such measure

$$s(x_n) = \frac{b(x_n) - a(x_n)}{\max(a(x_n), b(x_n))}$$

$a(x_n)$: average pairwise distance of x_n with points in its own cluster

$b(x_n)$: smallest average distance to points in another cluster.

Within-cluster sum of squares (WCSS) ($\text{WCSS} = \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{\mu}_{z_n}\|^2$)

Various other criteria, e.g., Davies–Bouldin and Calinski–Harabasz

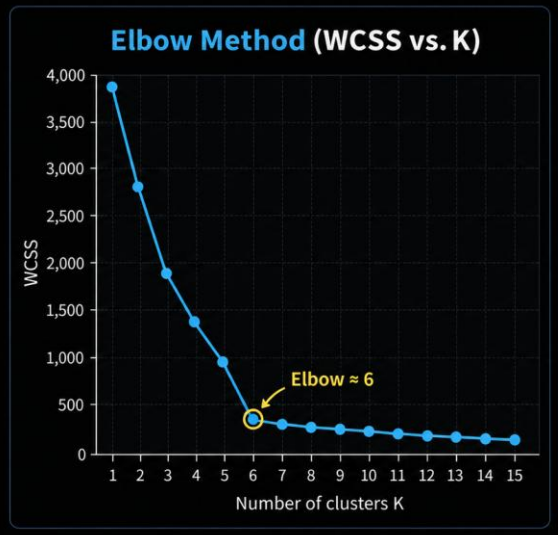
Performance on other tasks that use the output of a clustering algo can be a proxy to the clustering quality (extrinsic metrics)

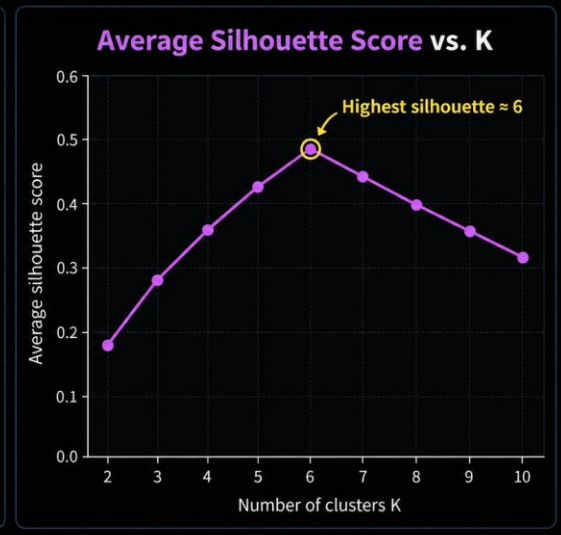
If labels are available, we can use those to assess labels' consistency with cluster ids given by the cluster algo (Purity, rand-index, etc)



K-means clustering: How to Choose K?

Do multiple runs with different K, plot evaluation metric versus K





K-means also has variants where K can be determined dynamically

K is a hyperparameter for any clustering algo that requires knowing the number of clusters (some probabilistic/Bayesian models provide principled ways to also learn K from data)

Clustering Algorithms Beyond K-means

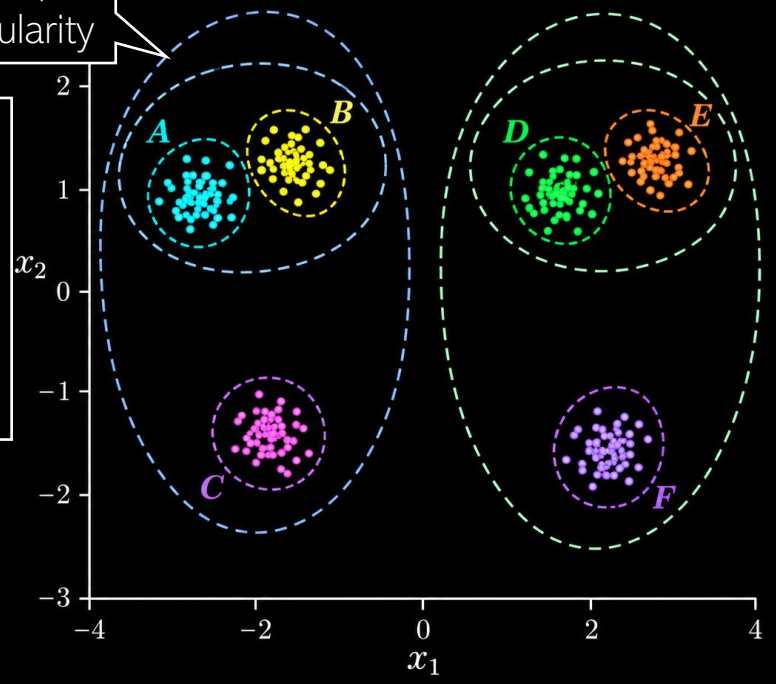
Cluster shape/geometry issues can be handled by variants of K-means

In some situations, however, even the best K-means variants may not be good enough or even applicable. Some examples below:

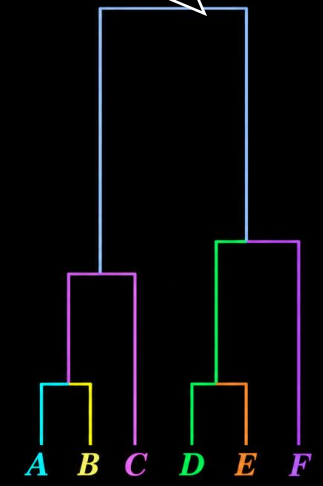
Clusters have a hierarchical structure

Clusters at multiple levels of granularity

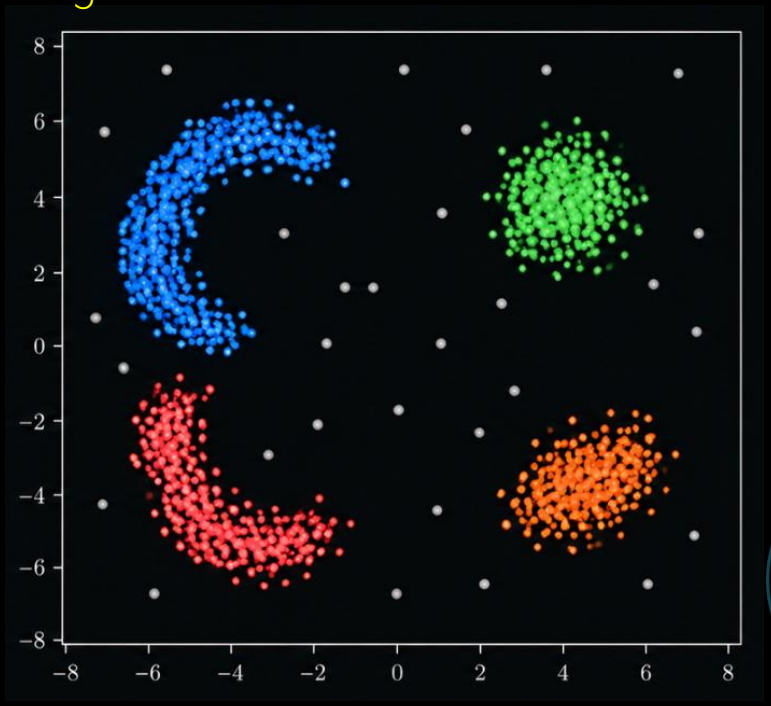
K-means or any other "flat" clustering algo won't be suitable



Dendrogram

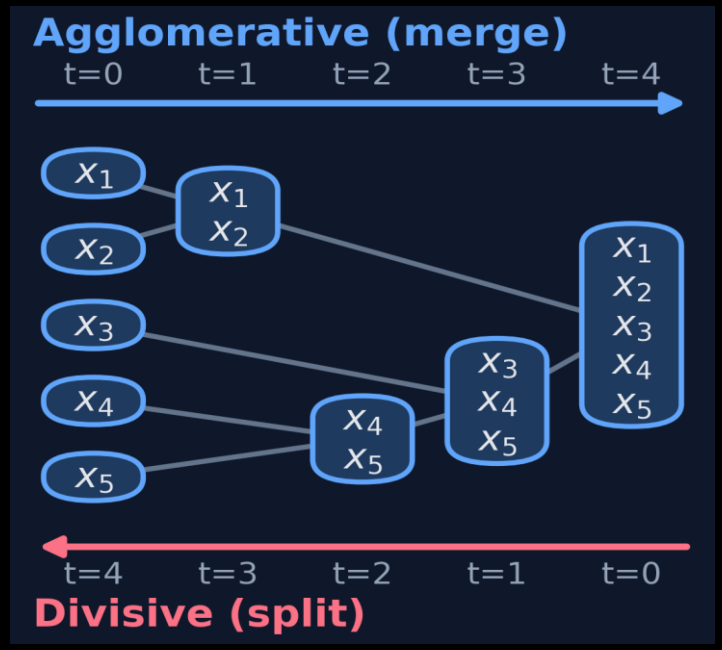


Data has several outliers in low-density region that should be left "unclustered"



Hierarchical Clustering

Primarily 2 ways to do HC: agglomerative and divisive

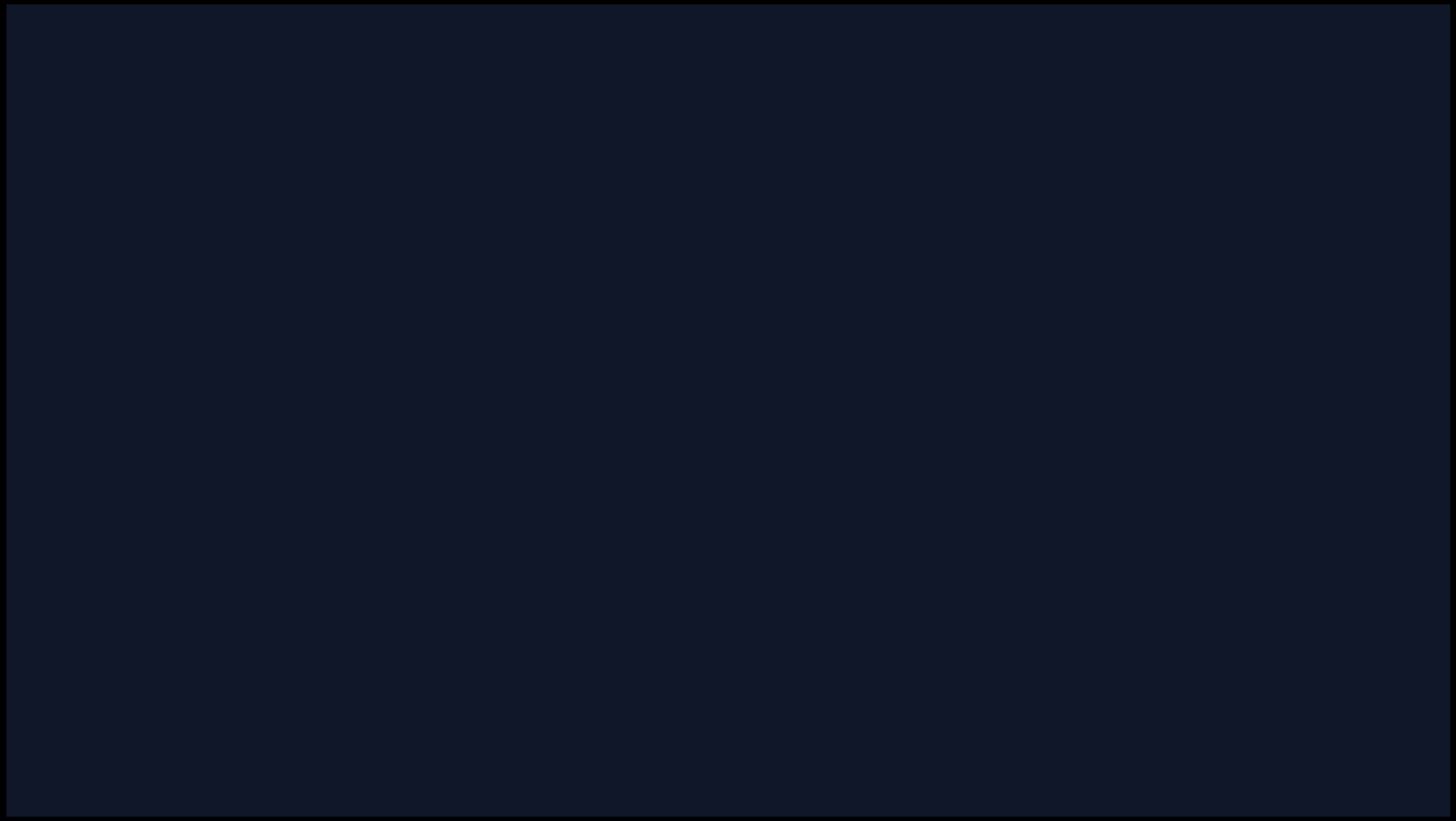


These algos don't need number of clusters to be specified

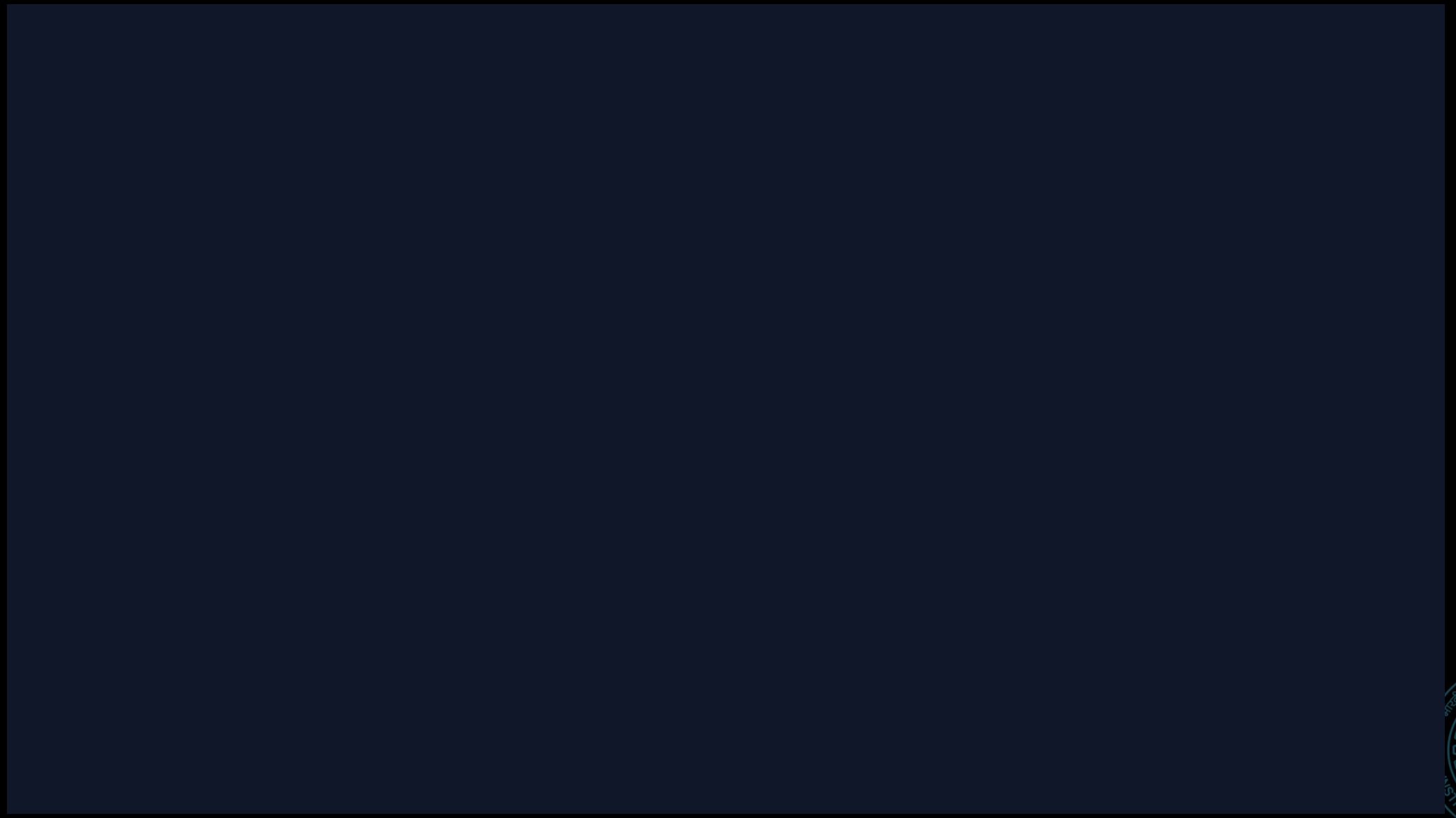
- We can cut the tree (dendrogram) at our desired level

Merge and split criteria, distance function, etc need to be chosen

Hierarchical Clustering: Agglomerative



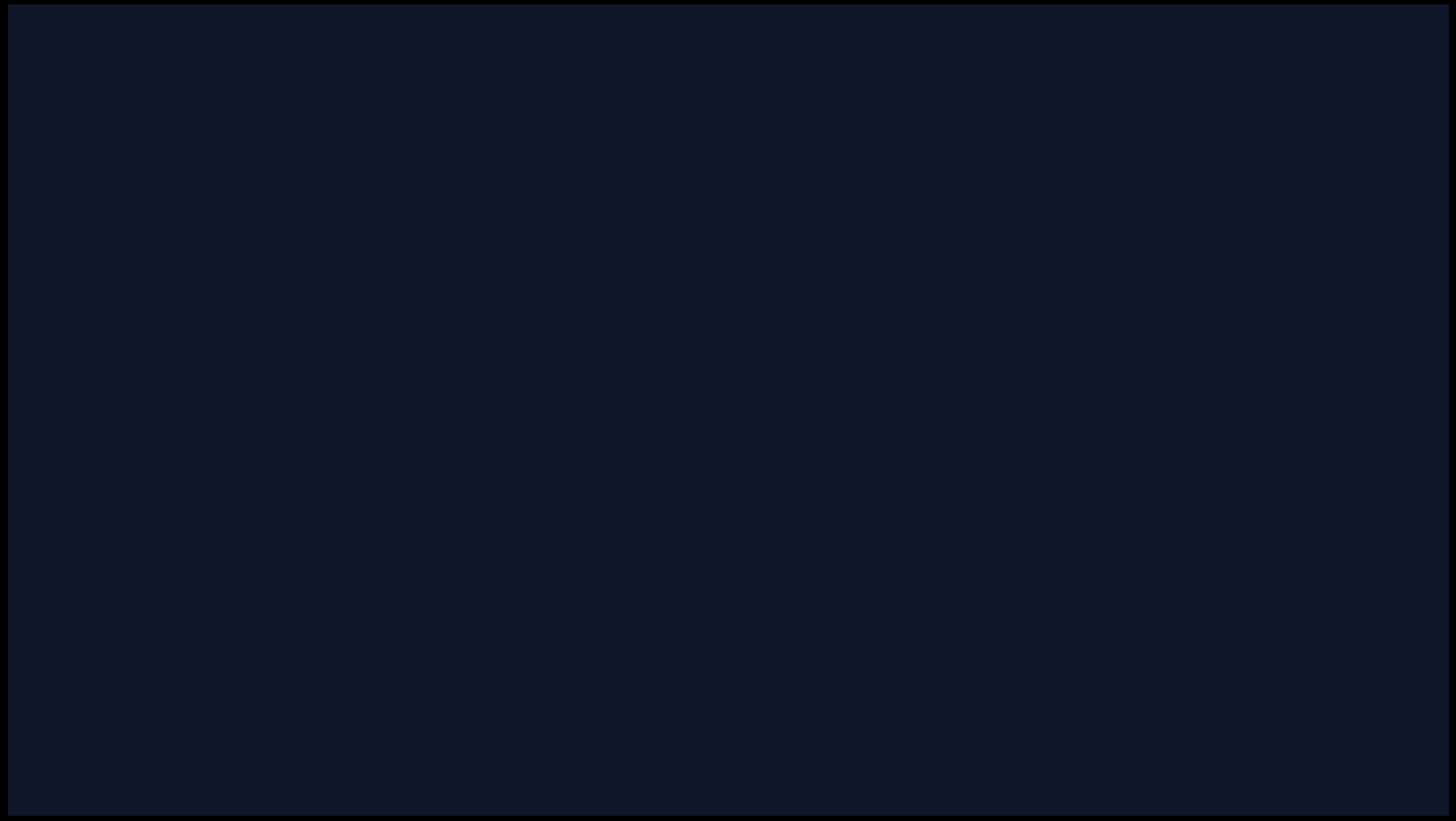
Hierarchical Clustering: Divisive



Density-based Clustering

"Connected points" make a cluster, outliers should not be clustered!

DBScan, a density-based clustering algo, does the same. Demo below



Overlapping Clustering

But still only one cluster assumed to be the true cluster

Hard clustering: z_n is one-hot. Soft clustering: z_n is probability vector

Overlapping Clustering: x_n may belong to multiple clusters simultaneously

In overlapping clustering, z_n is a binary vector (not the same as one-hot)

z_n = [1 0 0 1 0]

K=5 clusters with point x_n belonging (in whole, not in terms of probabilities) to clusters 1 and 4

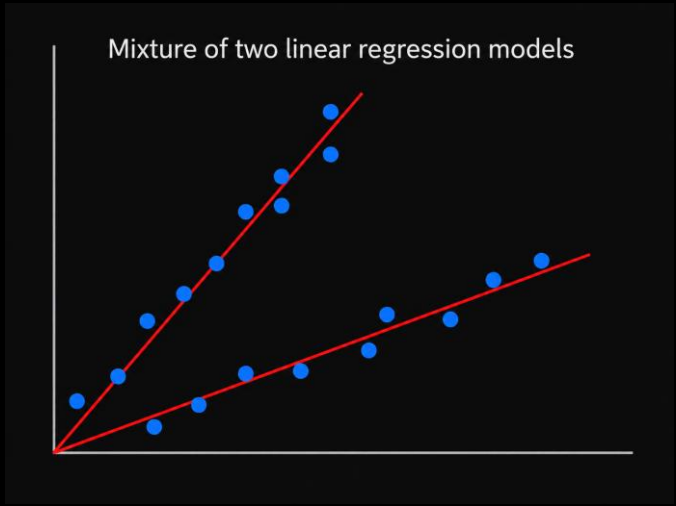
Overlapping clust. is similar to multi-label classification but unsupervised

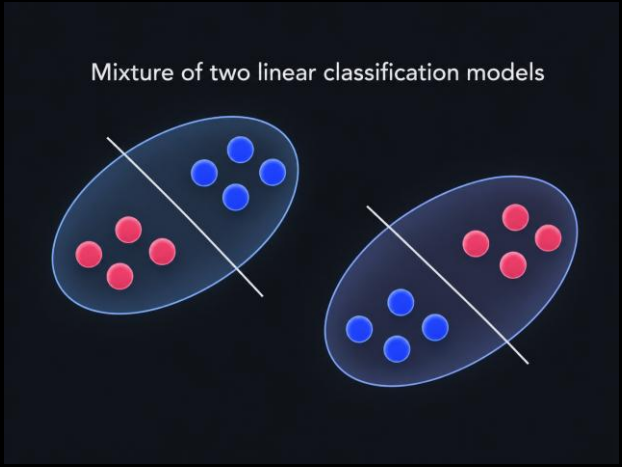
Some examples where overlapping clustering naturally arises:

- Clustering of genes that take part in several biological processes (each process represented by a cluster)
- Clustering movies based on genres (each genre represented by a cluster); a movie can be about multiple genres (action AND comedy)

Clustering can help supervised learning, too

Often "difficult" supervised learning problems can be seen as mixture of simpler (typically linear) models – clustering the inputs helps here





Such an approach is also an example of divide and conquer and is also known as "mixture of experts" (will see it more formally when we discuss latent variable models)



A simplified view of this approach is as follows:

$K = 2$ in examples above

- Initialize K simple models $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_K$ somehow (say random init)
- Assign each (x_n, y_n) to cluster $z_n = \arg\min_k \ell(\boldsymbol{w}_k, x_n, y_n)$
- Relearn $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_K$ using data assigned to their respective clusters

